

MACHINE LEARNING COURSE · CS-433

Optimization

September 9 + 15, 2026

Martin Jaggi

EPFL

Learning / Estimation / Fitting

Given a cost function $L(w)$, we wish to find $w^\star$ which minimizes the cost:

$$\min_{\mathbf{w}} \mathcal{L}(\mathbf{w}) \quad \text{subject to } \mathbf{w} \in \mathbb{R}^D$$

This means the *learning* problem is formulated as an **optimization problem**.

We will use an **optimization algorithm** to solve the problem (to find a good w).

Grid search

Grid search is one of the simplest optimization algorithms. We compute the cost over all values w in a grid, and pick the best among those.

This is brute-force, but extremely simple and works for any kind of cost function when we have very few parameters and the cost is easy to compute.

Grid search: the curse of dimensionality

For a large number of parameters D , however, grid search has too many “for-loops”, resulting in an exponential computational complexity:

If we decide to use 10 possible values for each dimension of w , then we have to check 10^D points. This is clearly impossible for most practical machine learning models, which can often have $D \approx$ millions of parameters. Choosing a good range of values for each dimension is another problem.

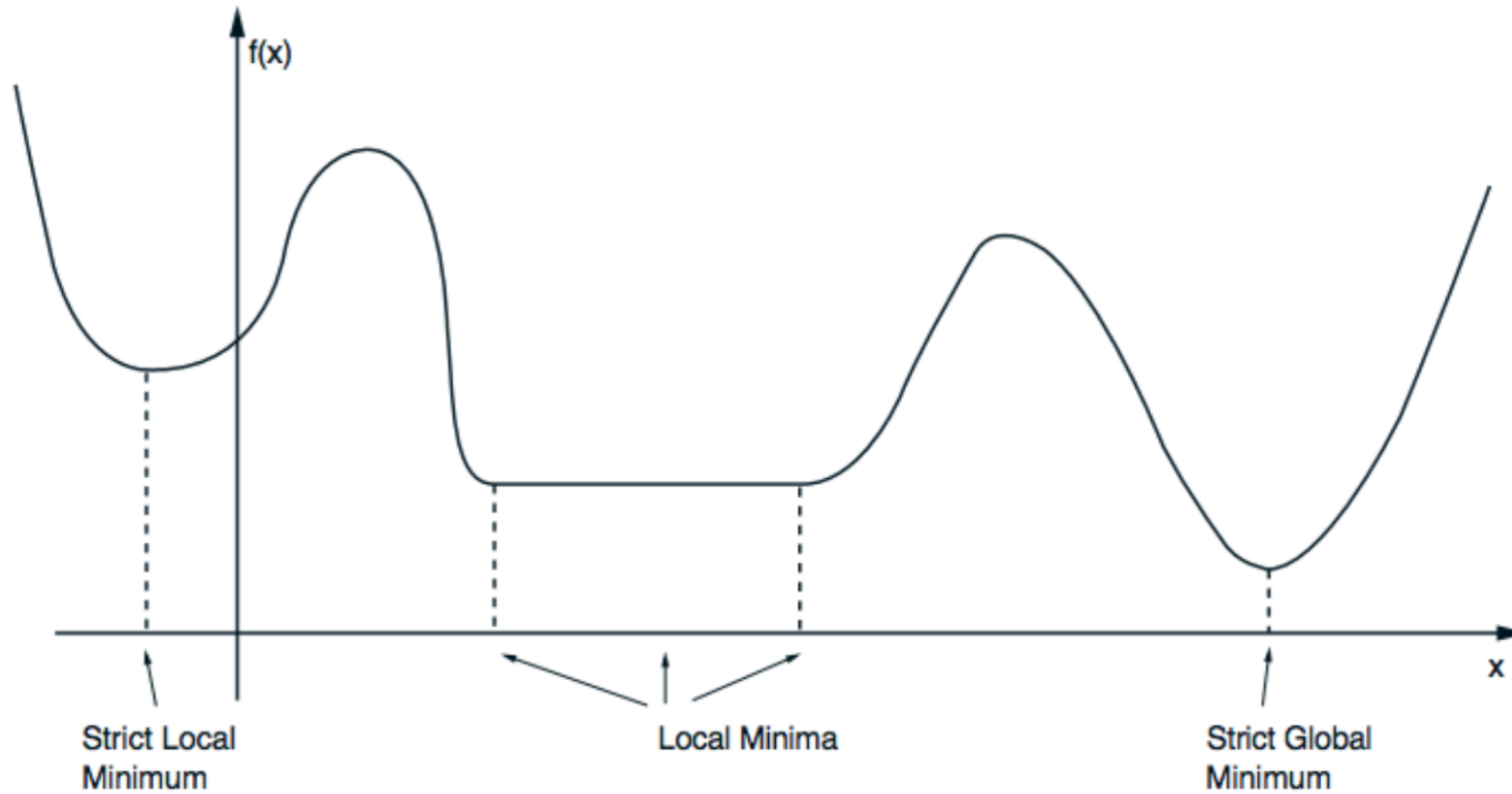
Other issues: No guarantee can be given that we end up close to an optimum.

1

Optimization landscapes

Where can an optimization algorithm end up, and what counts as a solution?

Optimization landscapes



The above figure is taken from Bertsekas, Nonlinear programming.

Local minimum

A vector $\mathbf{w}^\star$ is a **local minimum** of L if it is no worse than its neighbors; i.e. there exists an $\epsilon > 0$ such that,

$$\mathcal{L}(\mathbf{w}^\star) \leq \mathcal{L}(\mathbf{w}), \quad \forall \mathbf{w} \text{ with } \|\mathbf{w} - \mathbf{w}^\star\| < \epsilon$$

Global minimum

A vector $w^\star$ is a **global minimum** of L if it is no worse than all others,

$$\mathcal{L}(\mathbf{w}^\star) \leq \mathcal{L}(\mathbf{w}), \quad \forall \mathbf{w} \in \mathbb{R}^D$$

A local or global minimum is said to be **strict** if the corresponding inequality is strict for $w \neq w^\star$.

2

Smooth optimization

When the cost is differentiable, its gradient tells us where to go.

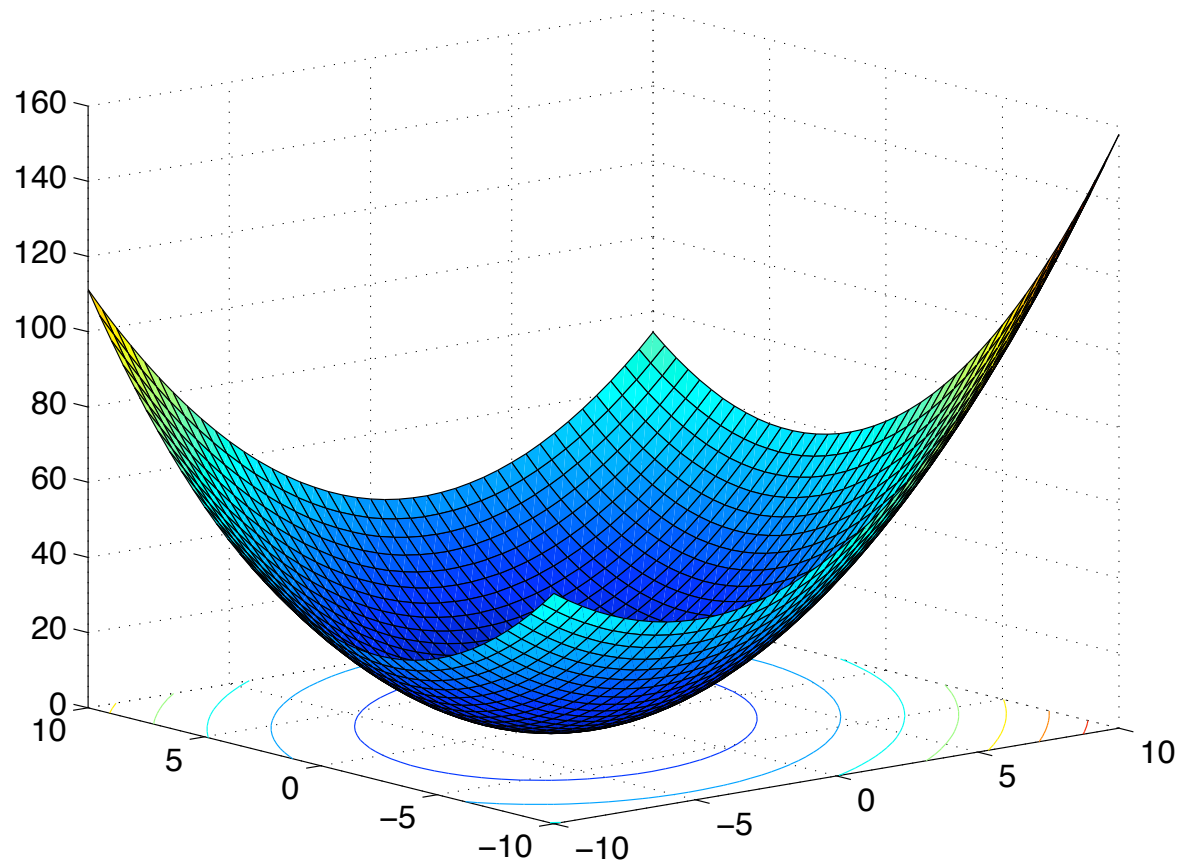
Follow the gradient

A gradient (at a point) is the slope of the tangent to the function (at that point). It points to the direction of largest increase of the function.

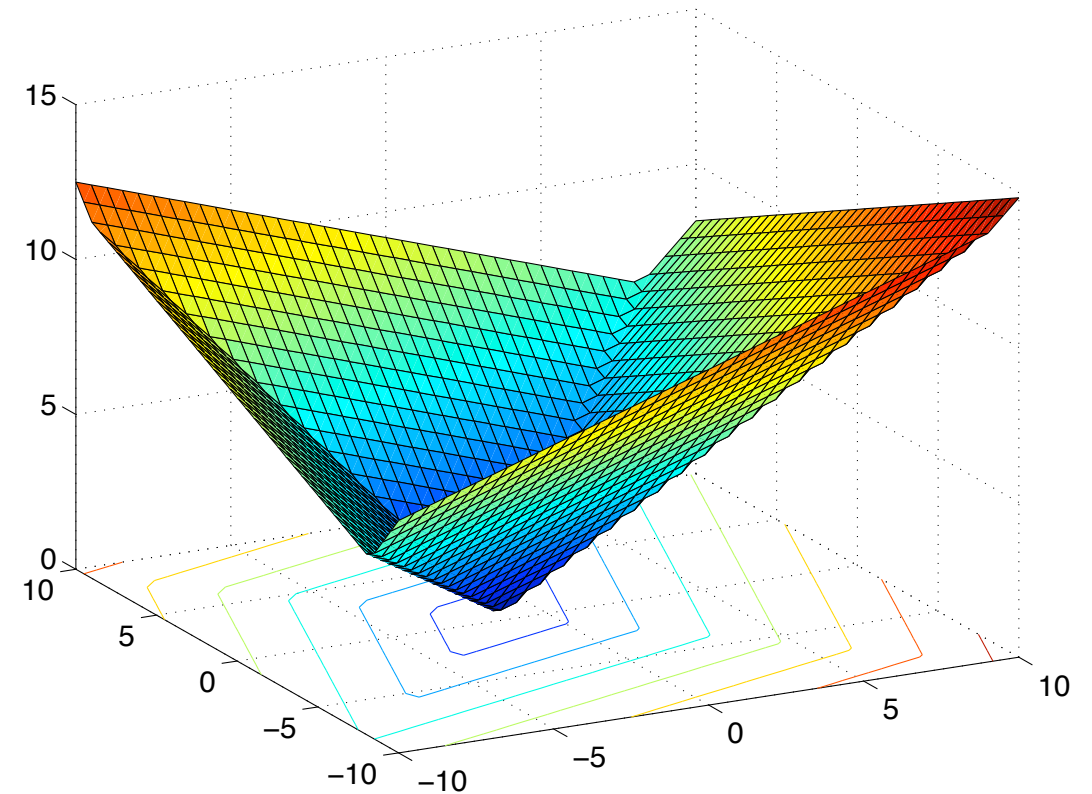
MSE and MAE for a 2-parameter model

(We used $y_n \approx w_0 + w_1 x_{n1}$ with $y^\top = [2, -1, 1.5]$ and $x^\top = [-1, 1, -1]$.)

MSE



MAE



Definition of the gradient

$$\nabla \mathcal{L}(\mathbf{w}) := \left[\frac{\partial \mathcal{L}(\mathbf{w})}{\partial w_1}, \dots, \frac{\partial \mathcal{L}(\mathbf{w})}{\partial w_D} \right]^\top$$

This is a vector, $\nabla \mathcal{L}(\mathbf{w}) \in \mathbb{R}^D$

Gradient descent

To minimize the function, we iteratively take a step in the (opposite) direction of the gradient

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \nabla \mathcal{L}(\mathbf{w}^{(t)})$$

where $\gamma > 0$ is the **step-size** (or **learning rate**). Then repeat with the next t .

Example

Gradient descent for a 1-parameter model to minimize MSE:

$$w_0^{(t+1)} := (1 - \gamma) w_0^{(t)} + \gamma \bar{y}$$

where $\bar{y} = \sum_n y_n / N$.

When is this sequence guaranteed to converge?

Gradient descent for linear MSE

For linear regression

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}, \quad \mathbf{X} = \begin{bmatrix} X_{11} & X_{12} & \dots & X_{1D} \\ X_{21} & X_{22} & \dots & X_{2D} \\ \vdots & \vdots & \ddots & \vdots \\ X_{N1} & X_{N2} & \dots & X_{ND} \end{bmatrix}$$

Gradient descent for linear MSE

We define the error vector $\mathbf{e}$:

$$\mathbf{e} = \mathbf{y} - \mathbf{X}\mathbf{w}$$

and MSE as follows:

$$\begin{aligned}\mathcal{L}(\mathbf{w}) &:= \frac{1}{2N} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 \\ &= \frac{1}{2N} \mathbf{e}^\top \mathbf{e}\end{aligned}$$

then the gradient is given by

$$\nabla \mathcal{L}(\mathbf{w}) = -\frac{1}{N} \mathbf{X}^\top \mathbf{e}$$

Computational cost

What is the complexity (# operations) of computing the gradient?

1. starting from w , and
2. given e and w ?

Variant with offset

Recall: alternative trick when also incorporating an offset term for the regression:

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}, \quad \tilde{\mathbf{X}} = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1D} \\ 1 & x_{21} & x_{22} & \dots & x_{2D} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N1} & x_{N2} & \dots & x_{ND} \end{bmatrix}$$

Stochastic gradient descent

Sum objectives

In machine learning, most cost functions are formulated as a **sum** over the training examples, that is

$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}_n(\mathbf{w})$$

where $\mathcal{L}_n$ is the cost contributed by the n -th training example.

Q: What are the $\mathcal{L}_n$ for linear MSE?

The SGD algorithm

The **stochastic gradient descent** (SGD) algorithm is given by the following update rule, at step t :

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \nabla \mathcal{L}_n(\mathbf{w}^{(t)})$$

Theoretical motivation

Idea: Cheap but unbiased **estimate** of the gradient!

In expectation over the random choice of n , we have

$$\mathbb{E}[\nabla \mathcal{L}_n(\mathbf{w})] = \nabla \mathcal{L}(\mathbf{w})$$

which is the true gradient direction. (check!)

Mini-batch SGD

There is an intermediate version, using the update direction being

$$\mathbf{g} := \frac{1}{|B|} \sum_{n \in B} \nabla \mathcal{L}_n(\mathbf{w}^{(t)}) \qquad \mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \mathbf{g}$$

In the above gradient computation, we have randomly chosen a subset $B \subseteq [N]$ of the training examples. For each of these selected examples n , we compute the respective gradient $\nabla \mathcal{L}_n$, at the same current point $\mathbf{w}^{(t)}$.

Mini-batch SGD in practice

The computation of g can be [parallelized](#) easily. This is how current deep-learning applications utilize GPUs (by running over $|B|$ threads in parallel).

Note that in the extreme case $B := [N]$, we obtain (batch) gradient descent, i.e. $g = \nabla L$.

SGD for linear MSE

See Exercise Sheet 2.

Computational cost

For linear MSE, what is the complexity (# operations) of computing the stochastic gradient?

(using only $|B| = 1$ data examples)

3

Variants of SGD

The same stochastic gradient, used in smarter ways.

SGD with momentum

pick a stochastic gradient $\mathbf{g}$

$$\mathbf{m}^{(t+1)} := \beta_1 \mathbf{m}^{(t)} + (1 - \beta_1) \mathbf{g} \quad (\text{momentum term})$$

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \mathbf{m}^{(t+1)}$$

- momentum from previous gradients (acceleration)

Adam

pick a stochastic gradient $\mathbf{g}$

$$\mathbf{m}^{(t+1)} := \beta_1 \mathbf{m}^{(t)} + (1 - \beta_1) \mathbf{g} \quad (\text{momentum term})$$

$$\mathbf{v}_i^{(t+1)} := \beta_2 \mathbf{v}_i^{(t)} + (1 - \beta_2) (\mathbf{g}_i)^2 \quad \forall i \quad (\text{2nd-order statistics})$$

$$\mathbf{w}_i^{(t+1)} := \mathbf{w}_i^{(t)} - \frac{\gamma}{\sqrt{\mathbf{v}_i^{(t+1)}}} \mathbf{m}_i^{(t+1)} \quad \forall i$$

- faster forgetting of older weights
- is a momentum variant of Adagrad
- coordinate-wise adjusted learning rate
- strong performance in practice, e.g. for neural networks

SignSGD

pick a stochastic gradient $\mathbf{g}$

$$\mathbf{w}_i^{(t+1)} := \mathbf{w}_i^{(t)} - \gamma \text{sign}(\mathbf{g}_i) \quad \forall i$$

- only use the sign (one bit) of each gradient entry $\rightarrow$ communication efficient for distributed training
- convergence issues

Muon

pick a stochastic gradient $\mathbf{G}$ (for a weight matrix $\mathbf{W}$)

$$\mathbf{M}^{(t+1)} := \beta_1 \mathbf{M}^{(t)} + (1 - \beta_1) \mathbf{G} \quad (\text{momentum term})$$

$$\mathbf{W}^{(t+1)} := \mathbf{W}^{(t)} - \gamma \text{msign}(\mathbf{M}^{(t+1)})$$

- same shape as SignSGD, with the entrywise sign replaced by its matrix counterpart
- msign sets every singular value of its input to 1, so the update has operator norm 1 and pushes equally hard in every direction
- in practice msign is computed by a Newton–Schulz iteration: 5 steps, matrix multiplications only, no SVD, and GPU-friendly
- applied only to the 2D weight matrices; embeddings, biases & gains and routers use AdamW, with step size $\gamma \approx 0.02$

4

Non-smooth optimization

What to do when the cost has kinks, as MAE does.

Convexity, again

An alternative characterization of *convexity*, for differentiable functions, is given by

$$\mathcal{L}(\mathbf{u}) \geq \mathcal{L}(\mathbf{w}) + \nabla \mathcal{L}(\mathbf{w})^\top (\mathbf{u} - \mathbf{w}) \quad \forall \mathbf{u}, \mathbf{w}$$

meaning that the function must always lie above its [linearization](#).

Subgradients

A vector $g \in \mathbb{R}^D$ such that

$$\mathcal{L}(\mathbf{u}) \geq \mathcal{L}(\mathbf{w}) + \mathbf{g}^\top (\mathbf{u} - \mathbf{w}) \quad \forall \mathbf{u}$$

is called a **subgradient** to the function L at w .

This definition makes sense for objectives L which are not necessarily differentiable (and not even necessarily convex).

If L is convex and differentiable at w , then the only subgradient at w is $g = \nabla L(w)$.

Subgradient descent

Identical to the gradient descent algorithm, but using a subgradient instead of gradient. Update rule

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \mathbf{g}$$

for $\mathbf{g}$ being a subgradient to L at the current iterate $\mathbf{w}^{(t)}$.

Example: optimizing linear MAE

1. Compute a subgradient of the absolute value function

$$h : \mathbb{R} \rightarrow \mathbb{R}, \quad h(e) := |e|$$

2. Recall the definition of the mean absolute error:

$$\mathcal{L}(\mathbf{w}) = \text{MAE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N |y_n - f_{\mathbf{w}}(\mathbf{x}_n)|$$

For linear regression, its (sub)gradient is easy to compute using the chain rule.
Compute it!

See Exercise Sheet 2.

Stochastic subgradient descent

Stochastic SubGradient Descent (still abbreviated SGD commonly).

Same, g being a subgradient to the randomly selected L_n at the current iterate $w^{(t)}$.

Exercise: Compute the SGD update for linear MAE.

Step-size and stopping

Step-size selection

If γ is too big, the method might diverge. If it is too small, convergence is slow. Convergence to a local minimum is guaranteed only when the step-size γ follows a schedule becoming small enough, where ‘small enough’ depends on the problem (landscape) parameters.

Stopping criteria

When $\nabla L(w)$ is (close to) zero, we are (often) close to the optimum value.

Feature normalization and pre-conditioning

Gradient descent is very sensitive to ill-conditioning, that is the phenomenon when some coordinates (directions) have change rates vastly different from others.

Therefore, it is typically advised to normalize input features if possible, and also normalize each layer in a neural network (we will get to this in the later chapters on neural networks). Without this, step-size selection is much more difficult since different “directions” might converge (or fail to do so) at different speed.

Rescaling the space in this sense is also known as *pre-conditioning* the optimization problem.

Optimality conditions

For a *smooth* optimization problem, the first-order *necessary* condition says that at *an* optimum the gradient is equal to zero. Points of zero gradient are called **critical points**.

$$\nabla \mathcal{L}(\mathbf{w}^*) = 0$$

If L is convex and $w^\star$ is a critical point, then $w^\star$ is a global optimum.

The Hessian

We can use the second derivative to study if a candidate point is a local minimum (not a local maximum or saddle-point) using the Hessian matrix,

$$\nabla^2 \mathcal{L}(\mathbf{w}) := \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w} \partial \mathbf{w}^\top}(\mathbf{w})$$

The second-order *sufficient* condition states that if

- $\nabla L(w) = 0$ (critical point)
- and $\nabla^2 L(w) > 0$ (positive definite),

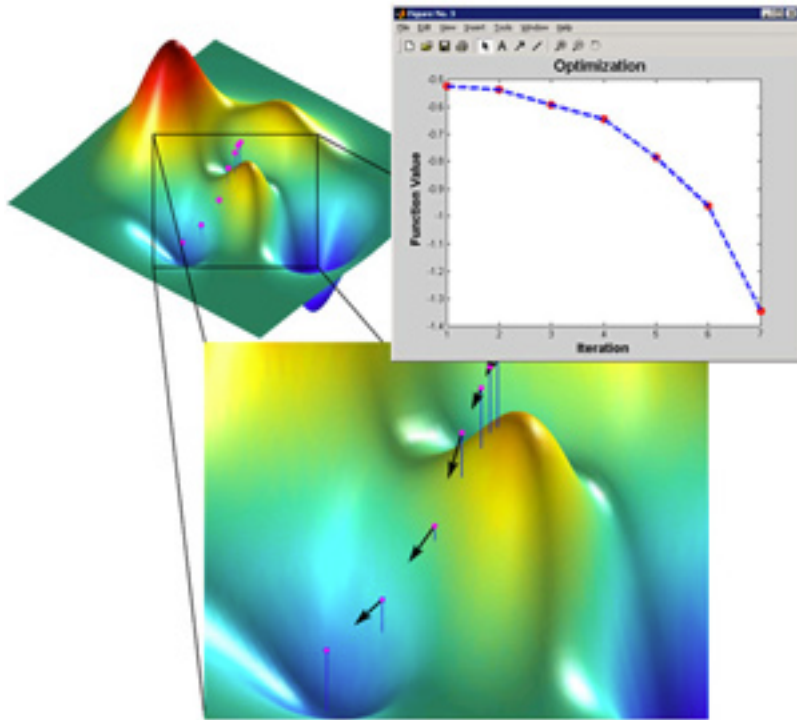
then w is a local minimum. The Hessian is also related to convexity: a twice-differentiable function is convex if and only if the Hessian is positive semi-definite at all points.

5

Non-convex optimization

Everything so far assumed the nice case. Reality is bumpier.

Non-convex optimization



Real-world problems are **not convex**!

All we have learnt on algorithm design and performance of convex algorithms still helps us in the non-convex world.

*image from mathworks.com

6

Constrained optimization

When the answer is only allowed to live in part of the space.

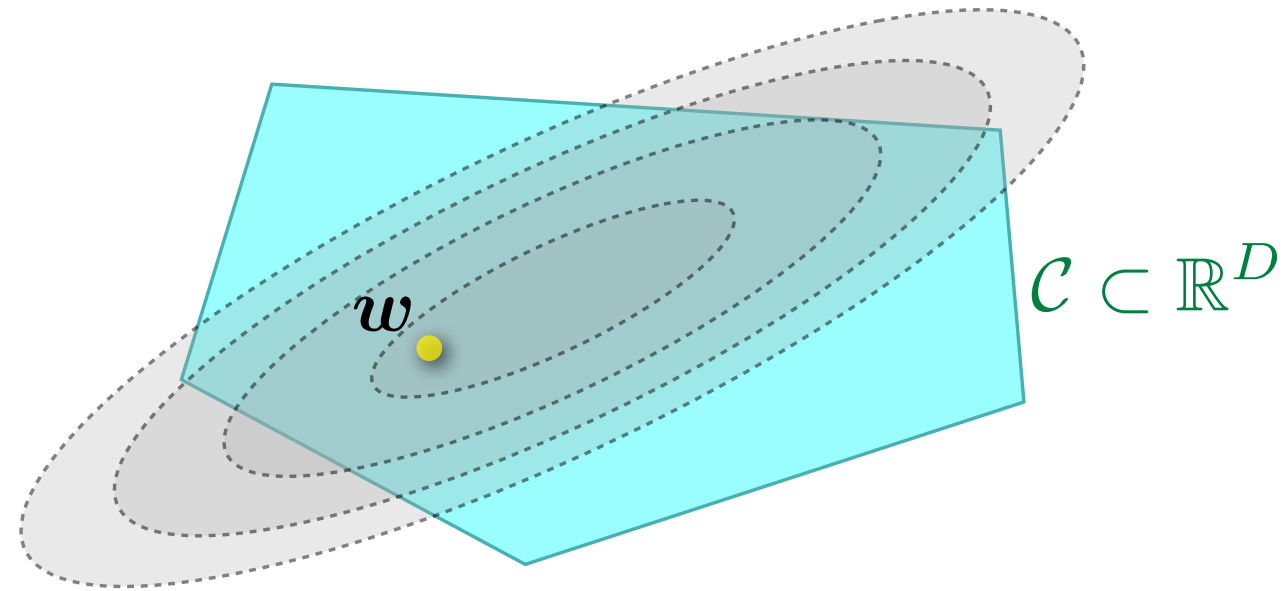
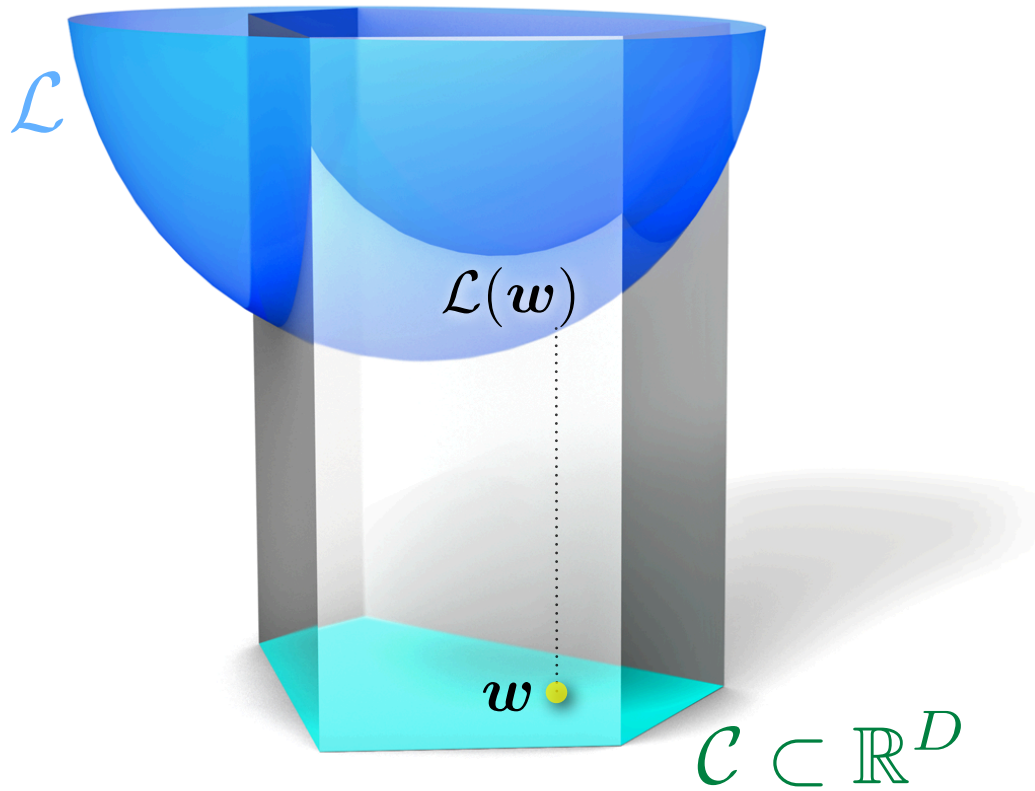
Constrained optimization

Sometimes, optimization problems come posed with additional constraints:

$$\min_{\mathbf{w}} \mathcal{L}(\mathbf{w}), \quad \text{subject to } \mathbf{w} \in \mathcal{C}$$

The set $\mathcal{C} \subset \mathbb{R}^D$ is called the **constraint set**.

Constrained optimization



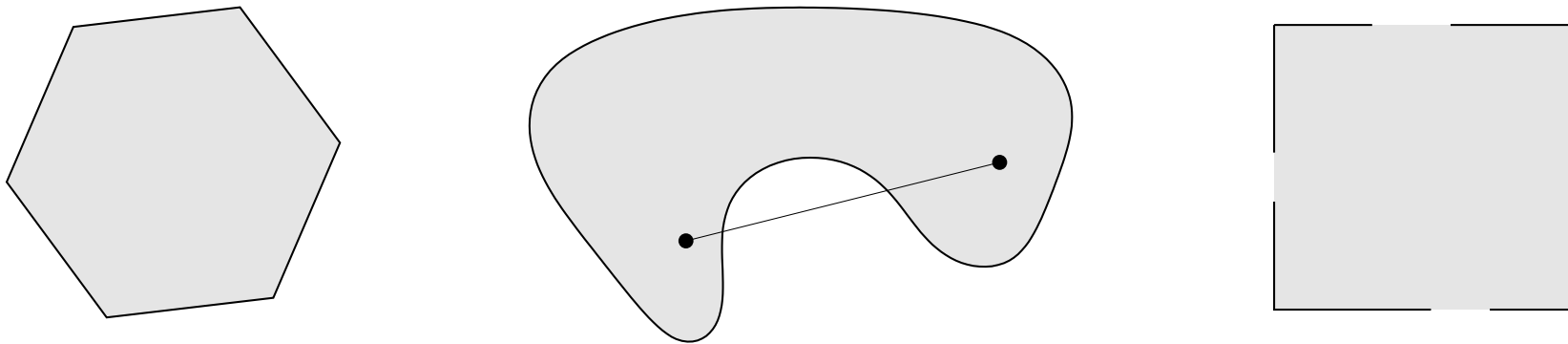
Solving constrained optimization problems

- A) Projected Gradient Descent
- B) Transform it into an *unconstrained* problem

Convex sets

A set C is **convex** iff the line segment between any two points of C lies in C , i.e., if for any $u, v \in C$ and any θ with $0 \leq \theta \leq 1$, we have

$$\theta \mathbf{u} + (1 - \theta) \mathbf{v} \in C$$



*Figure 2.2 from S. Boyd, L. Vandenberghe

Properties of convex sets

- Intersections of convex sets are convex
- Projections onto convex sets are unique (and often efficient to compute).

Formal definition:

$$P_{\mathcal{C}}(\mathbf{w}') := \arg \min_{\mathbf{v} \in \mathcal{C}} \|\mathbf{v} - \mathbf{w}'\|$$

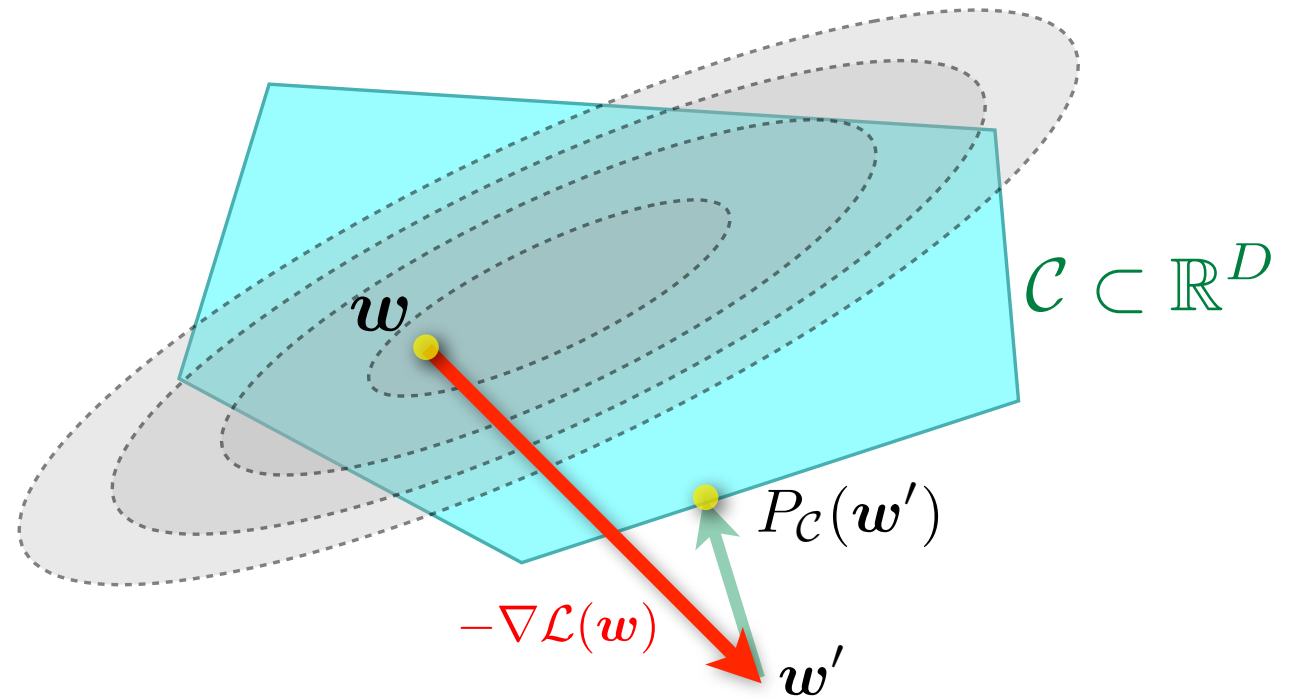
Projected gradient descent

Idea: add a **projection onto $\mathcal{C}$** after every step:

$$P_{\mathcal{C}}(\mathbf{w}') := \arg \min_{\mathbf{v} \in \mathcal{C}} \|\mathbf{v} - \mathbf{w}'\|$$

Update rule:

$$\mathbf{w}^{(t+1)} := P_{\mathcal{C}}[\mathbf{w}^{(t)} - \gamma \nabla \mathcal{L}(\mathbf{w}^{(t)})]$$



Projected SGD

Same SGD step, followed by the projection step, as above.
Same convergence properties.

Computational cost of projection? Crucial!

Turning constrained into unconstrained problems

(Alternatives to projected gradient methods)

Use **penalty functions** instead of directly solving $\min_{w \in C} L(w)$.

- “*brick wall*” (indicator function)

$$l_C(\mathbf{w}) := \begin{cases} 0 & \mathbf{w} \in C \\ \infty & \mathbf{w} \notin C \end{cases} \Rightarrow \min_{\mathbf{w} \in \mathbb{R}^D} \mathcal{L}(\mathbf{w}) + l_C(\mathbf{w})$$

(disadvantage: non-continuous objective)

- Penalize error. *Example:*

$$C = \{\mathbf{w} \in \mathbb{R}^D \mid A\mathbf{w} = \mathbf{b}\} \Rightarrow \min_{\mathbf{w} \in \mathbb{R}^D} \mathcal{L}(\mathbf{w}) + \lambda \|A\mathbf{w} - \mathbf{b}\|^2$$

- Linearized penalty functions (see Lagrange multipliers)

7

Additional notes

(Not exam material)

Grid search and hyper-parameter optimization

Read more about grid search and other methods for “hyperparameter” setting:

en.wikipedia.org/wiki/Hyperparameter_optimization#Grid_search

Computational complexity

The **computation cost** is expressed using the **big- $\mathcal{O}$** notation. Let f and g be two functions defined on some subset of the real numbers. We write $f(x) = \mathcal{O}(g(x))$ as $x \rightarrow \infty$, if and only if there exists a positive real number c and a real number x_0 such that

$$|f(x)| \leq c |g(x)|, \quad \forall x > x_0$$

- What is the computational complexity of matrix multiplication?
- What is the computational complexity of matrix-vector multiplication?

SGD theory

When N is large, choosing a random training example (x_n, y_n) and taking an SGD step is advantageous:

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma^{(t)} \nabla \mathcal{L}_n(\mathbf{w}^{(t)})$$

For convergence, $\gamma^{(t)} \rightarrow 0$ “appropriately”. One such condition, called the Robbins–Monro condition, suggests to take $\gamma^{(t)}$ such that:

$$\sum_{t=1}^{\infty} \gamma^{(t)} = \infty, \quad \sum_{t=1}^{\infty} (\gamma^{(t)})^2 < \infty$$

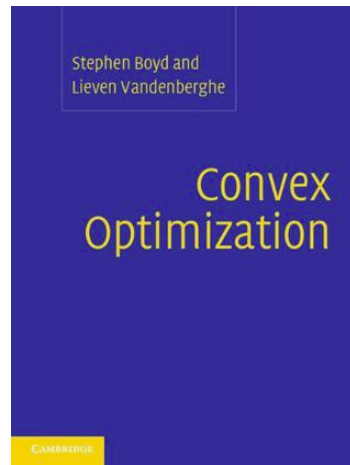
One way to obtain such sequences is $\gamma^{(t)} := 1/(t+1)^r$ where $r \in (0.5, 1)$

More optimization theory

If you want, you can gain a deeper understanding of several optimization methods relevant for machine learning from this survey:

Convex Optimization: Algorithms and Complexity — by Sébastien Bubeck

And also from the **book of Boyd & Vandenberghe** (both are free online PDFs)



Exercises



1. Chain-rule. If it has been a while, familiarize yourself with it again.
2. Revise computational complexity ($\mathcal{O}$ -notation).
3. Derive the computational complexity of grid-search, gradient descent and stochastic gradient descent for linear MSE (# steps and cost per step).
4. Derive the gradients for the linear MSE and MAE cost functions.
5. Implement gradient descent and gain experience in setting the step-size.
6. Implement SGD and gain experience in setting the step-size.